Cancel Send

This is a **TypeScript utility for Express** that helps you handle errors in **async route handlers** cleanly.

Let's break it down step by step.

---

1. The problem it solves

In Express, **errors thrown in async functions are NOT automatically caught**.

❌ Without this helper:

ts Copy code

```ts
app.get("/users", async (req, res) => {
  const users = await getUsers(); // throws error
  res.json(users);
});
// ❌ Express will NOT catch the error
```

You'd normally have to write `try/catch` in every route:

ts Copy code

```ts
app.get("/users", async (req, res, next) => {
  try {
    const users = await getUsers();
    res.json(users);
  } catch (err) {
    next(err);
  }
});
```

This gets repetitive and messy.

---

2. The imports

ts Copy code

```ts
import { NextFunction, Request, Response } from "express";
```

These are **TypeScript types** from Express:

- `Request` → HTTP request
- `Response` → HTTP response
- `NextFunction` → function to pass control/errors to Express

---

3. The `AsyncHandler` type

ts Copy code

```ts
type AsyncHandler = (
  req: Request,
  res: Response,
  next: NextFunction
) => Promise<void>
```

This defines the **shape of an async Express route handler**:

- Takes `req`, `res`, `next`
- Returns a `Promise` (because it's `async`)

Example that matches this type:

ts Copy code

```ts
const handler: AsyncHandler = async (req, res) => {
  res.send("Hello");
};
```

---

4. The `catchAsync` function

ts Copy code

```ts
export const catchAsync = (fn: AsyncHandler) =>
  (req: Request, res: Response, next: NextFunction) => {
```

This is a **higher-order function**:

- It **takes an async function (`fn`)**
- Returns a **new Express middleware function**

Think of it as a **wrapper**.

---

5. Wrapping the async function

ts Copy code

```ts
Promise.resolve(fn(req, res, next)).catch((err) => {
  next(err)
})
```

What's happening:

1. `fn(req, res, next)` runs your async route handler
2. `Promise.resolve(...)` ensures it's treated as a promise
3. If the promise **rejects or throws an error**:
   - `.catch(err => next(err))` forwards the error to Express
4. Express then sends the error to your **global error handler**

---

6. How you use it

Before ❌

ts Copy code

```ts
router.get("/users", async (req, res, next) => {
  try {
    const users = await getUsers();
    res.json(users);
  } catch (err) {
    next(err);
  }
});
```

After ✅

ts Copy code

```ts
router.get(
  "/users",
  catchAsync(async (req, res) => {
    const users = await getUsers();
    res.json(users);
  })
);
```

No `try/catch` , cleaner code.

---

7. Why this pattern is popular

- ✔ Removes repetitive `try/catch`
- ✔ Centralizes error handling
- ✔ Works with Express's error middleware
- ✔ Cleaner, more readable routes

This pattern is used in many production Express apps.

---

8. One-sentence summary

`catchAsync` **is a wrapper that automatically catches errors in async Express route**

handlers and passes them to Express's error-handling middleware.

If you want, I can also show:

- how this compares to `express-async-errors`
- how to type it more flexibly
- or how error middleware receives these errors